

Large Language Models and Generative AI

Lecture 12 – Introduction to Modern Machine Learning

Steven Campbell

Imperial College London Summer School

June 2026

Outline

1. From Transformers to generation
2. Training and adaptation
3. Reliability and grounding
4. AI workflows and responsible use

From Transformers to generation

Where this lecture fits

Earlier lectures introduced:

- loss functions, generalisation, validation and regularisation;
- dimension reduction and learned representations;
- online learning and sequential feedback;
- multilayer perceptrons, convolutional networks and Transformer blocks.

Lecture 11 built the language-model machinery:

text $\rightarrow$ tokens $\rightarrow$ embeddings $\rightarrow$ attention $\rightarrow$ contextual representations.

Today

We complete the path to **next-token generation**, then study how models become assistants and how assistants are embedded in reliable, responsible workflows.

What is generative AI?

Generative model

A generative model represents a distribution over possible outputs, often conditioned on an input:

$$p_{\theta}(y \mid x).$$

It can generate new samples from that distribution.

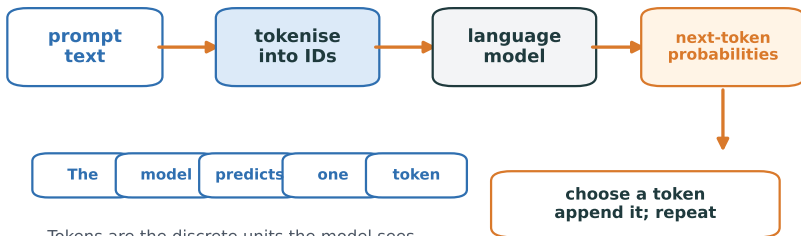
Generative AI includes systems for text, code, images, audio, video and synthetic data. Many systems are **multimodal**: several kinds of input or output may be represented as tokens.

Today's detailed case

An autoregressive language model generates text by repeatedly computing a **next-token distribution**, selecting one token, appending it, and repeating.

Next-token generation

From text to generated text



Tokens are the discrete units the model sees.
The model outputs a distribution over the vocabulary.

Language modelling: the key factorisation

For a token sequence $x_1, \dots, x_T$, the chain rule gives

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}), \quad x_{<t} = (x_1, \dots, x_{t-1}).$$

Next-token pretraining objective

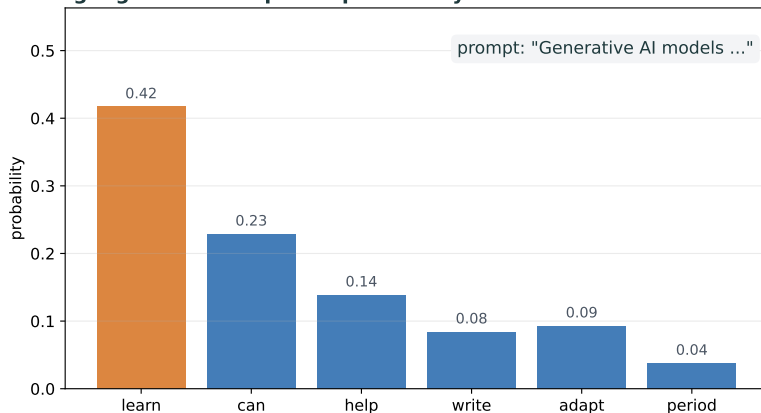
Make every observed continuation likely:

$$\min_{\theta} - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}).$$

- This is **self-supervised learning**: shifting the sequence supplies a target at every position.
- During training, causal masking prevents access to future tokens even though losses across positions can be computed in parallel.

Next-token prediction

A language model outputs a probability distribution for the next token



The model does not output a complete answer in one step: it outputs one score for every possible next token.

Autoregressive generation

Autoregressive generation: append one token at a time



At each step the model conditions on the tokens retained in the current context window.

Efficient implementations reuse cached computations, but the output tokens are still produced sequentially.

Conceptual algorithm

Start with a context. Compute the next-token distribution. Choose one token. Append it. Repeat until a stopping condition is reached.

Next-token generation: animated view

Each step conditions on the tokens already generated.

Sampling choices

Let s_i be the logit for token i .

Greedy decoding

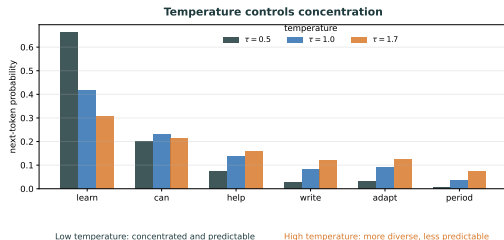
$$x_{t+1} = \arg \max_i s_i.$$

Temperature sampling

$$p_i^{(\tau)} = \frac{\exp(s_i/\tau)}{\sum_j \exp(s_j/\tau)}, \quad \tau > 0.$$

- lower τ : more concentrated;
- higher τ : more varied;

Sampling choices influence output variability.



Mini-exercise: generation choices

A model sees The medicine should be taken ... and assigns:

token	daily	with	after	only	other
p	0.35	0.25	0.20	0.10	0.10

Discuss:

- 1 What would greedy decoding select?
- 2 Why can sampling be useful for creative writing?
- 3 Why would neither greedy decoding nor low temperature make the medical instruction trustworthy?
- 4 What evidence and review would be required before anyone acted on it?

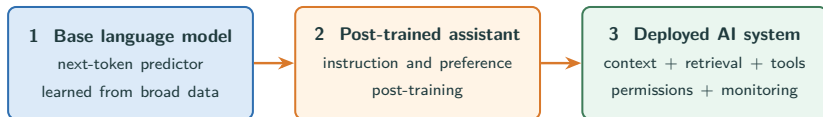
Takeaway

A decoding rule chooses among continuations; it does not establish the truth or safety of the content.

Training and adaptation

From a base model to an AI system

A model is not the whole AI system



The application assembles a context such as:



The user's visible message is only one part of the information and controls surrounding the model.

Stage 1: pretraining

For an autoregressive language model, pretraining repeatedly learns

$$x_{<t} \mapsto p_{\theta}(x_t \mid x_{<t}).$$

- Training data are broad and are not usually organised around one downstream task.
- Good prediction across many domains can require reusable representations of syntax, semantics, code, factual associations, etc.
- The learned parameters are not a verified database: associations may be incomplete, biased or stale.

Why rich behaviour can emerge

Predicting well over long, varied contexts can require tracking relations, goals and likely continuations.

Key limitation

The loss rewards observed continuations. Truth is learned indirectly; it is not separately certified.

Stage 2: instruction tuning

A base model learns to continue text. A useful assistant should respond more reliably to tasks, roles and requested formats.

Instruction tuning uses examples of the form

instruction + context $\longrightarrow$ desired response.

Base objective

Continue the supplied text in a statistically plausible way.

Post-trained behaviour

Follow instructions, respect formats and adopt interaction conventions more reliably.

Examples include summarising, translating, explaining, classifying, structured output and code generation.

Stage 3: preference, safety and outcome post-training

User-facing models are often adapted further using comparisons, feedback or task-specific rewards.

- ① generate candidate responses;
- ② score them with human judgements, model-based feedback or automatic task checks;
- ③ update the model to favour responses that better satisfy the chosen criteria.

What this can improve

Helpfulness, tone, formatting, refusals, instruction following and performance on tasks with useful feedback signals.

What it does not guarantee

Perfect factuality, faithful reasoning, safety in every context or agreement about contested trade-offs.

Methods include reinforcement learning from feedback, direct preference objectives and rewards from verifiable outcomes.

Prompting and in-context learning

Prompts condition the current next-token distribution.

In-context learning

A model can sometimes infer a task from instructions or examples placed in the prompt:

```
English:  cat   →  French:  chat
English:  dog   →  French:  chien
English:  horse →  French:  ?
```

- The examples change the current context, not the model weights.
- Any apparent learning is temporary unless the surrounding application stores information separately.
- The prompt may contain system instructions, conversation history, retrieved sources and tool results as well as the user's visible message.

What LLMs are good at and what remains hard

Often useful for

- drafting and rewriting;
- summarising text;
- explaining concepts;
- generating code sketches;
- brainstorming and translation.

Still risky for

- unsupported factual claims;
- precise citations or numbers without verification;
- high-stakes advice;
- hidden assumptions;
- tasks needing evidence outside the prompt.

Reliability and grounding

AI sounds convincing, but is it correct?

Fluency is not the same as truth

fluent claim, no evidence

**The wording sounds plausible,
but the claim is unsupported.**

risk: fabricated facts, citations or numbers

better-grounded draft

**Claims are linked to checked
sources and uncertainty is visible.**

goal: claim + source + uncertainty + review

Retrieval makes evidence available; it does not prove that the evidence is correct, complete or interpreted faithfully.

Common failure modes

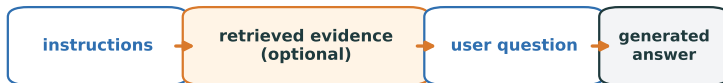
- **Hallucinated facts:** confident claims that are false or unsupported.
- **Fake citations:** plausible-looking references that do not exist or do not support the claim.
- **Outdated knowledge:** the world may have changed after the model was trained.
- **Prompt sensitivity:** small wording changes can change the output.
- **Hidden bias:** outputs may reflect patterns and imbalances in training data.

Practical rule

For factual or high-stakes work, treat an LLM answer as a draft or hypothesis until independently checked.

Context windows: what the model can directly see

Context window: what the model can directly see



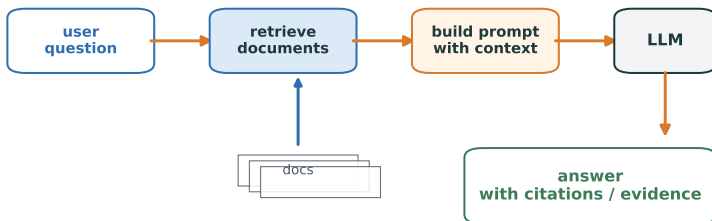
The model directly conditions only on tokens placed in the current context.

If the needed evidence is absent, the model may rely on memorised patterns or guess.

RAG helps by retrieving evidence and inserting it into the context window before generation.

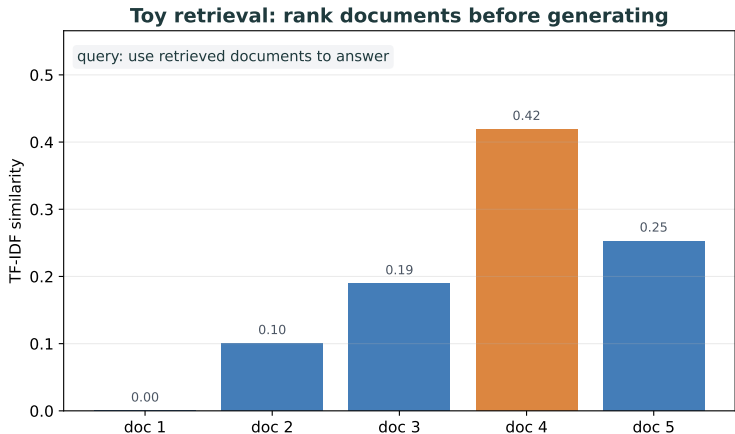
Retrieval-augmented generation

Retrieval-augmented generation (RAG)



RAG helps use current/private information, but it depends on retrieving the right evidence.

Toy retrieval example



Idea

Rank candidate documents first; pass the top evidence to the language model as context.

RAG: strengths and limitations

RAG is useful because it puts retrieved evidence into the model's context window before generation.

Why RAG helps

- Adds current or private information.
- Can provide evidence and citations.
- Reduces reliance on model memory.
- Separates search from generation.

What can still fail

- The retriever may miss the right document.
- Retrieved text may be wrong or incomplete.
- The model may misread evidence.
- Citations may not support every claim.

RAG improves grounding; it does not remove the need for evaluation.

Evaluating generated answers

There is no single perfect metric for generative AI. Evaluation depends on the task.

Criterion	Example question
Factuality	Are claims supported by evidence?
Task success	Did the output solve the user's problem?
Robustness	Does it behave similarly under small prompt changes?
Calibration	Does it express uncertainty when appropriate?
Safety	Could this output cause harm if followed?
Reproducibility	Can the result be checked or regenerated?

For high-stakes uses

Prefer task-specific tests, human review, source checking and monitoring over casual inspection.

AI workflows and responsible use

A human-in-the-loop workflow

A human-in-the-loop AI workflow



Use the model to accelerate work; keep humans responsible for goals, checks and final decisions.

Prompting as interface design

- 1 **task** What should the model do?
- 2 **context** What information should it use?
- 3 **constraints** What should it avoid?
- 4 **format** How should the answer look?
- 5 **checks** How should uncertainty be handled?

**Good prompts reduce ambiguity.
They do not replace evidence or verification.**

A practical prompt pattern

A useful prompt often contains task, context, constraints and output format.

Template

Task: Summarise the text below for a nontechnical audience.

Context: The audience has seen basic ML but not this paper.

Constraints: Do not invent numbers. Flag uncertainties.

Format: Five bullet points plus one sentence
on limitations.

Text: ...

AI workflows for coding and data analysis

Useful workflow pattern:

- ① Ask for a plan or pseudocode before code.
- ② Run code in a controlled environment.
- ③ Check shapes, units, data leakage and train/test separation.
- ④ Write small tests or sanity checks.
- ⑤ Inspect plots and numerical outputs.
- ⑥ Treat the model's explanation as a draft, not proof.

Verification loop

Ask for a draft → extract the claims or tests → check them independently
→ revise.

Responsible use checklist

Before relying on an AI output, ask...

- 1 evidence** What supports each important claim?
- 2 stakes** What happens if this is wrong?
- 3 privacy** Am I disclosing sensitive or restricted data?
- 4 fairness** Who could be affected or excluded?
- 5 agency/security** What can the system read, call, change or disclose?
- 6 traceability** Can we reconstruct sources, prompts, versions and human edits?

Higher stakes?

**Use trusted evidence.
Test independently.
Restrict permissions.
Document uncertainty.
Require qualified approval.**

Practical question

Would you be comfortable explaining exactly how this output was produced and checked? For factual claims, keep a record of citations and sources!

Summary

- ① Autoregressive language models learn a next-token distribution from the available context.
- ② An output projection maps the final contextual vector to vocabulary logits; generation repeatedly selects and appends tokens.
- ③ Token probability, fluent language and cautious wording are not reflections of truth.
- ④ Pretraining and post-training shape model behaviour; a deployed system additionally supplies context, retrieval, tools, permissions and monitoring.
- ⑤ Prompting, retrieval, tools and fine-tuning solve different problems and introduce different failure modes.
- ⑥ Trustworthy use requires relevant evidence, independent checks, restricted permissions, traceability and proportionate human responsibility.

Closing idea

LLMs have become incredibly powerful, but a well-designed workflow is needed to ensure that they lead to useful, trustworthy and responsible outcomes.